# Multimodal Fashion & Context Retrieval

An intelligent search engine that retrieves images from a diverse fashion dataset given natural-language descriptions. It aims to understand not just what someone is wearing, but where they are and the overall vibe of the attire.

**Repository:** https://github.com/buddywhitman/multimodal-fashion-retrieval

## 1. Approaches

There are several ways to build natural-language to image retrieval for fashion. Each fits a different situation, and the tradeoffs matter.

| Approach | How it works | Good for | Weakness |
|---|---|---|---|
| **Keyword / filename matching** | Match query words against filenames or manual tags | Trivial to stand up | No generalization. Can't handle synonyms or unseen phrasings |
| **Trained multi-label classifier + filter** | Train a CNN on a fixed taxonomy, filter images by predicted labels | High precision on labels seen in training | No zero-shot ability. Open vocabulary ("bright yellow raincoat") and scene or vibe language are out of reach, and adding a concept means retraining |
| **Vanilla CLIP, single global embedding** | Encode whole image and whole query into one shared vector space, rank by cosine similarity | Genuinely zero-shot, strong on scene, style, and vibe language | Pooling a whole image into one vector loses compositional structure. "Red tie with a white shirt" and "white tie with a red shirt" embed almost identically. Color is also fuzzy |
| **Region-based detection + captioning** | Detect each region, caption it, embed each caption separately | Best possible fine-grained, compositional accuracy | Needs a detector at index time. Heavier pipeline and more failure surface than the problem warrants |
| **Hybrid: fashion CLIP (whole-image + per-garment region) + structured attributes from segmentation, re-ranked (chosen)** | Whole-image CLIP for scene and vibe, plus per-garment crops and mask-derived (category, color) attributes for exact compositional matching | Zero-shot and compositional at once, using annotations the dataset already ships | Structured signal is bounded by the category vocabulary and by what exists in the corpus. Both are measured, not assumed |

The real tension here is zero-shot generalization versus compositional precision. Vanilla CLIP has the first and lacks the second. A trained classifier has the second and lacks the first. The chosen approach keeps CLIP's zero-shot behavior as a fallback for everything, then layers exact, instance-scoped attribute matching on top for the compositional cases where CLIP alone breaks down.

## 2. Chosen Approach

### 2.1 Dataset

The corpus is Fashionpedia. It uses the val2020 split plus a sampled subset of the larger train2020 split, for a total of 15,189 images and 110,907 annotated garment instances. Both splits share Fashionpedia's instance-segmentation format: per-garment polygons over a 46-category taxonomy. train2020 is included so the evaluation queries' attribute combinations, such as a yellow coat or a red tie, have real ground truth to retrieve. Sampling takes every train2020 image containing the rarer target categories, then adds a reproducible random sample of the rest for diversity. The corpus spans all three required axes: environment (office, street, park, home, studio, runway, beach), clothing type (formal, casual, outerwear, sporty), and a wide color palette.

### 2.2 Architecture

**Part A - Indexer** ( `indexer/` )

1. **`dataset.py`** loads and merges the Fashionpedia annotations into plain per-image records. Each garment instance carries its category label and segmentation mask.
2. **`embed.py`** embeds with **`patrickjohncyh/fashion-clip`** , a CLIP model fine-tuned on roughly 800K fashion image/text pairs. It grounds garment types, colors, and styles far better than general-purpose CLIP, and it runs on GPU when one is available. Two representations are produced per image: the whole image, which captures scene and overall style, and each garment cropped from its bounding box and embedded on its own. That second representation is the key to compositionality (see 2.3).
3. **`color_extract.py`** and **`common/colors.py`** derive each garment's color from its segmentation mask. The mask is eroded inward to avoid background bleed, the median RGB is taken over a cropped region, and the result is named by nearest-neighbor search in CIE Lab space, which is perceptually uniform. The reference palette comes from the XKCD color survey, roughly 800 human color-name judgments. Chromatic families (blue, navy, denim, cornflower blue, and so on) are derived programmatically, so a query for "blue" earns full credit for an exact match and graded credit for a family member. A second color is recorded when a garment is genuinely multi-colored.
4. **`relations.py`** and **`depth_relations.py`** record which garments are layered together, using bounding-box overlap within a body region, and then which garment sits on top. A monocular depth model runs only on those candidate pairs to resolve the z-order.
5. **`scene_tag.py`** tags each image's scene (place type), style, and weather zero-shot, by cosine-matching the image embedding against small natural-language prompt banks.
6. **`build_index.py`** stores whole-image embeddings and all structured metadata in one Chroma collection, and per-garment crop embeddings in another. Chroma is a single pip install, persistent, with metadata filtering built in. That keeps the engineering light so the effort goes into ML logic rather than storage plumbing.

**Part B - Retriever** ( `retriever/` )

1. **`query_parser.py`** turns free text into structured signals: (category, color) garment pairs, scene/style/weather terms, and layering relations with direction ("jacket over shirt"). Multi-word color names like "marine blue" are matched as single tokens. Garment words outside the known vocabulary are resolved zero-shot (see 2.4).

2. **`search.py`** combines whole-image CLIP similarity (Chroma HNSW) with deterministic exact recall. For each requested (category, color-family) it pulls every image containing a matching garment through an in-memory attribute index, then scores each candidate:

   `score = W_CLIP * image_similarity + W_COMP * attribute_overlap + W_SCENE * (scene/style/weather match)`

renormalized over whichever signals the query actually produced. `attribute_overlap` gives 1.0 for an exact per-garment color match, 0.8 for a same-family match, and 0.6 for category-only, plus a bonus when a requested layering direction matches the stored z-order. The weights (0.20, 0.70, 0.10) are chosen by ablation (see 3.2). If a query produces no structured signal, it falls back to pure CLIP similarity. The hybrid machinery only ever sharpens results. It never blocks a zero-shot query.

### 2.3 How it handles fashion queries

**Compositionality** is the hard part. Consider "red tie with a white shirt" against "white tie with a red shirt". A single pooled image embedding contains "red", "white", "tie", and "shirt" for both phrasings, so it cannot tell them apart. This system embeds each garment crop on its own and re-ranks with exact, instance-scoped (category, color) attributes from the segmentation masks. "Tie = red" and "shirt = white" are bound to specific garments and cannot bleed into each other. A controlled color-swap experiment measures this directly. Whole-image CLIP separates the correct composition from the swapped one at 0.60 accuracy, barely above chance. The hybrid reaches 1.00, with a mean margin of +0.38.

**Fine-grained color** benefits from Lab-space naming over an 800-name palette. It distinguishes the shades fashion queries lean on: burgundy against maroon, navy against denim, khaki against olive. Chromatic-family matching means a broad color word still recalls all its specific shades.

**Context** covers the "where" and the "vibe". The whole-image embedding, together with zero-shot scene, style, and weather tags, handles "inside a modern office", "casual weekend outfit", and "a rainy day". Adding a location or style term to a query measurably tightens the results (see 3.3).

**Layering** is answered with real z-order. "A jacket over a shirt" is a directed relation, not just co-occurrence.

### 2.4 Zero-shot capability

This works at two levels. First, the whole-image CLIP embedding handles descriptions that match no explicit label. "An elegant evening gown for a gala" surfaces dress, formal, and runway images even though none of those words is a stored label. Second, the query parser is itself zero-shot for garment vocabulary. Unknown words like "windbreaker", "parka", or "loafers" are resolved through fashion-CLIP with a two-prototype classifier. It accepts a word as a garment only if the word is more garment-like than place-like, action-like, person-like, or time-like, which avoids false matches on function words. On a held-out test this sits at 8/8 recall and 3/3 precision.

## 3. Benchmarking & Profiling

Everything below is reproducible from `eval/`. Ground truth is Fashionpedia's own segmentation and category labels, read back from the built index, so relevance is judged against real corpus attributes rather than hand-labeling.

### 3.1 The 5 evaluation queries

Precision@5, with color relevance judged by chromatic family to match how the retriever scores:

| # | Query | P@5 |
|---|---|---|
| 1 | "A person in a bright yellow raincoat." | 1.00 |
| 2 | "Professional business attire inside a modern office." | 1.00 |
| 3 | "Someone wearing a blue shirt sitting on a park bench." | 1.00 |
| 4 | "Casual weekend outfit for a city walk." | 1.00 |

| | | |
|---|---|---|
| 5 | "A red tie and a white shirt in a formal setting." | 1.00 |
| | **mean** | **1.00** |

### 3.2 Corpus-grounded benchmark

Five prompts are too small a sample to trust. So `eval/benchmark.py` auto-generates a query for every (category, color) combination that occurs at least three times in the corpus, then checks whether the true-positive images land in the top five.

| Slice | # queries | mean P@5 | mean R@5 |
|---|---|---|---|
| **GARMENT** (wearable clothing types) | 2,010 | **0.817** | **0.913** |
| PART (hardware and embellishments, harder and out of scope per PRD) | 1,630 | 0.793 | 0.870 |
| ALL | 3,640 | 0.806 | 0.894 |

P@5 is bounded by the metric itself. About 34% of combinations have fewer than five true positives, so their P@5 caps below 1.0 no matter what. A combination with three true positives caps at 3/5, which is 0.60. The mean achievable ceiling on this benchmark is 0.888. The GARMENT slice at 0.817 sits close to it, and R@5 at 0.913 is near its own ceiling.

The scoring weights come from a sweep ( `eval/ablate_weights.py` ) over GARMENT combos:

| (W_CLIP, W_COMP, W_SCENE) | mean P@5 | mean R@5 |
|---|---|---|
| 0.70 / 0.20 / 0.10 (CLIP-heavy) | 0.433 | 0.485 |
| 0.35 / 0.55 / 0.10 | 0.785 | 0.889 |
| **0.20 / 0.70 / 0.10 (chosen)** | **0.805** | **0.916** |
| 0.10 / 0.80 / 0.10 | 0.805 | 0.916 |

Exact (category, color) matches are ground truth from segmentation. Once a candidate is known to contain the right garment and color, that is a more reliable ranking signal than fuzzy CLIP similarity, so the optimum leans on the compositional term. W_CLIP = 0.20 is the plateau knee. It captures the full gain while keeping CLIP weight at twice the scene weight, a margin that protects pure-scene queries, and pure-scene ranking is verified unchanged across the whole sweep.

### 3.3 Context awareness (color + type + location)

`eval/multi_attribute.py` builds the literal three-attribute query for real (category, color, scene) combinations, then compares it to the same query with the location term removed.

| Query form | mean P@5 |
|---|---|
| color + type only | 0.29 to 0.38 |
| **color + type + location** | **0.59 to 0.69** |

| | |
|---|---|
| **effect of the location term** | **+0.30** |

The location term roughly doubles precision. A given color and garment is often shared across many scenes, and the scene score resolves the ambiguity. The effect reproduces consistently across independent samples.

### 3.4 Backbone selection

`eval/compare_backbones.py` puts the chosen `patrickjohncyh/fashion-clip` against the larger `Marqo/marqo-fashionCLIP` on the compositional-discrimination task.

| Backbone | Discrimination accuracy | Mean margin |
|---|---|---|
| **patrickjohncyh/fashion-clip (chosen)** | 0.650 | +0.006 |
| Marqo/marqo-fashionCLIP | 0.633 | +0.006 |

The larger checkpoint does not outperform the chosen one here, so the smaller and faster model is used. The embedding module supports both HuggingFace-native and OpenCLIP-native checkpoints, so swapping the backbone is a one-line config change.

### 3.5 Latency profile

Steady-state timings on the 15,189-image and 99,300-crop corpus:

| Query type | median latency |
|---|---|
| single garment + color | 32 ms |
| two garments + style | 33 ms |
| scene/style only | 30 ms |
| high-recall (large color family) | 34 ms |

Query cost is dominated by the fashion-CLIP text encode, around 15 to 20 ms. Exact recall and candidate scoring run against in-memory indexes: one maps (category, color) to image ids, the other caches image metadata and embeddings. Both are built once at startup, in about six seconds and tens of megabytes, which turns per-query lookup and scoring into dict and numpy operations instead of repeated database scans.

### 3.6 Scalability to 1M images

Per-query cost stays flat as the corpus grows. The image ANN pulls a fixed-size candidate pool through Chroma HNSW, which is sub-linear, and exact recall plus scoring are dict and numpy lookups. The in-memory indexes trade startup memory for query speed. That trade is negligible at 15K images. At around 1M the image-embedding store would reach a couple of gigabytes, at which point it moves to Chroma's server or sharded mode. The retriever already isolates that behind the standard collection API plus two swappable in-memory helpers, so it is a bounded change rather than a rewrite. Indexing throughput is GPU-accelerated, and color extraction works on garment crops rather than full images.

## 4. Codebase (GitHub) Link

**https://github.com/buddywhitman/multimodal-fashion-retrieval**

The repository is organized as:

- `common/` - shared configuration and the color classifier
- `indexer/` - Part A: feature extraction and vector storage
- `retriever/` - Part B: query parsing, search logic, and in-memory indexes
- `eval/` - coverage checks, benchmarks, ablations, and regression tests, all independently reproducible
- `docs/` - this write-up and the original PRD

Setup, dataset fetch, indexing, query, and evaluation instructions are in `README.md`.

# 5. Approaches for Future Work

## 5.1 Adding locations (cities, places) and weather

Place type and weather are already inferred zero-shot. The design uses place type (office, park, beach, cafe, gym) rather than city names on purpose, because a street-style photo does not reliably reveal which city it was taken in. There are a few natural extensions.

Finer place granularity, such as mall, restaurant patio, or subway platform, is just more prompts in the banks in `common/config.py`. The tagging and scoring machinery generalizes to any added axis with no architecture change.

Real deployment metadata is the bigger lever. Where EXIF, GPS, or capture-timestamp data exists, it should be preferred over pixel inference. Geolocation gives true city and place. Timestamp joined with location against a weather API gives true weather. That turns "locations and weather" from an inference problem into a metadata join, which is strictly more reliable, and it slots into the same (image to tag) metadata fields the retriever already scores against.

A dedicated scene classifier, for example Places365, could also stand in for or complement the CLIP-prompt tagging if higher scene accuracy is needed.

## 5.2 Improving precision

A learned color namer, trained on labeled color data, would replace nearest-neighbor over a fixed palette. The Lab-space infrastructure is already the right foundation. Only the reference points would become learned.

A learned re-ranker over the three scoring signals (image similarity, attribute overlap, scene/style/weather) would replace the fixed weights with a model trained on relevance-labeled queries. That can capture interactions a single global weight vector cannot.

Full z-order and true "tucked in" reasoning would go beyond pairwise over and under. It needs an occlusion signal, from instance-segmentation overlap direction or a trained relation classifier, rather than relative depth alone.

A stronger or more specialized backbone can be adopted as such checkpoints appear. Because the embedding module already supports both HuggingFace and OpenCLIP formats, evaluating and adopting one is a config change plus a benchmark run.

Finally, corpus growth helps the rarest attribute combinations. Long-tail retrieval quality is ultimately bounded by how many true positives exist to retrieve, and more source data addresses that directly.